



CS-343: Web Technologies

Dr. Naima Iltaf

Tenured Professor

Naima.iltaf@seecs.edu.pk

Lecture-6

Spring-2026

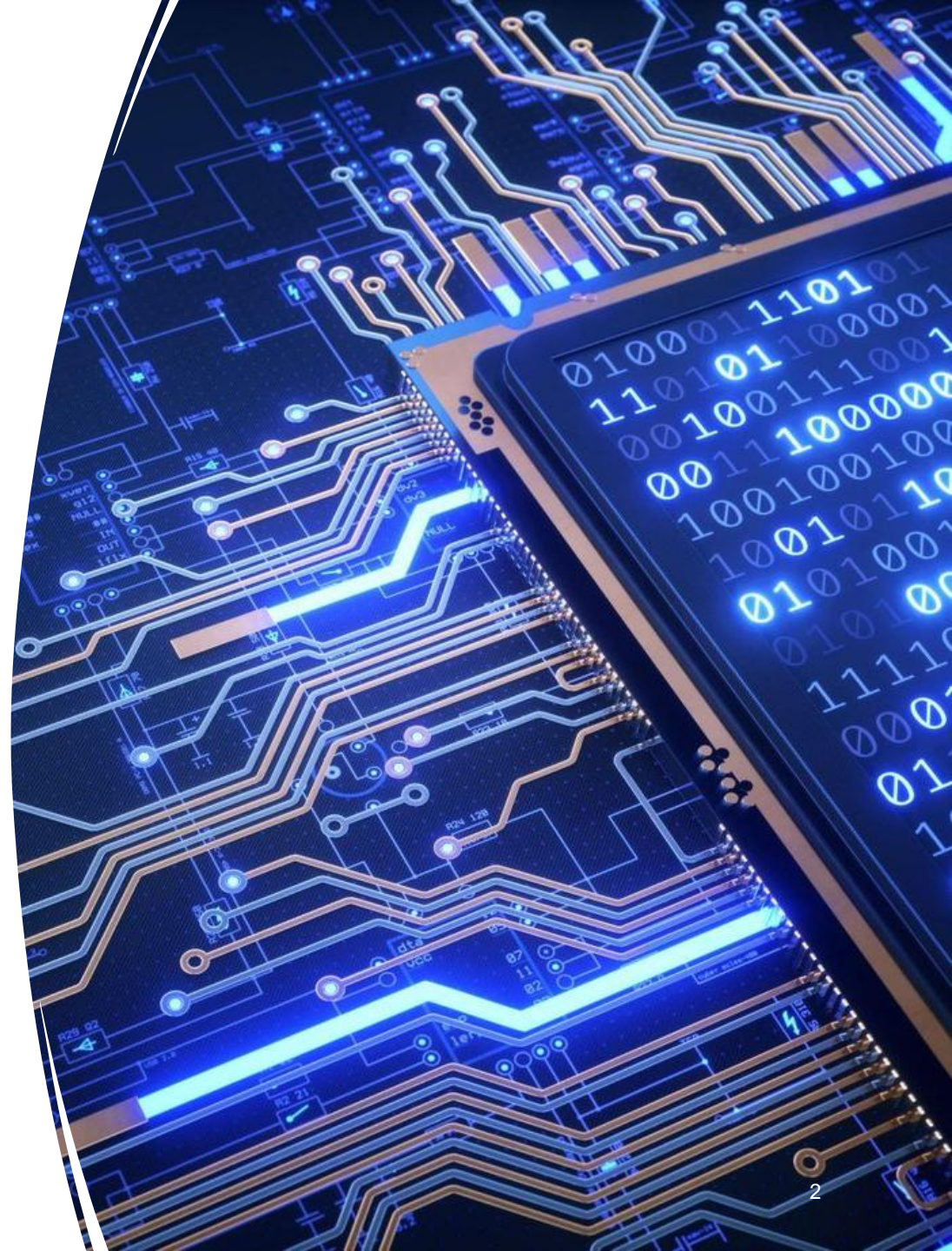
**School of Electrical Engineering and
Computer Science**

**National University of Science and
Technology(NUST)**

Agenda

JavaScript

- Functions
- arrays
- DOM
- Events



Defining Functions

block of code that performs a specific task, can be invoked whenever needed

Function Definition

```
function functionName( ) {
```

```
    //do some work
```

```
}
```

```
function functionName( param1, param2 ...) {
```

```
    //do some work
```

```
}
```

Function Call

```
functionName( );
```

```
1    //Function -> 2 numbers, sum
2
3    function sum(x, y) {
4        s = x + y;
5        return s;
6    }
7
8    let val = sum(3, 4);
9    console.log(val);
10
```

Arrow Functions

Compact way of writing a function

```
const functionName = ( param1, param2 ...) => {
```

```
//do some work
```

```
}
```

A variable that stores a function

```
const sum = ( a, b ) => {
```

```
    return a + b;
```

```
}
```

```
1  //sum function
2  function sum(a, b) {
3      return a + b;
4  }
5  //Modern JS
6  const arrowSum = (a, b) => {
7      console.log(a + b);
8  };
```


forEach Loop in Arrays

arr.forEach(callbackFunction)

CallbackFunction : Here, it is a function to execute for each element in the array

**A callback is a function passed as an argument to another function.*

```
arr.forEach( ( val ) => {  
    console.log(val);  
})
```

forEach Loop in Arrays

```
arr.forEach( callbackFunction )
```

CallbackFunction : Here, it is a function to execute for each element in the array

**A callback is a function passed as an argument to another function.*

```
arr.forEach( ( val ) => {  
    console.log(val);  
} )
```

forEach is a METHOD (not a normal function)

- Function → independent (stands alone)
- Method → belongs to an object

forEach is an array method that takes a **callback function** as its parameter

Example

```
let numbers = [1,2,3,4];  
  numbers.push(5);
```

Is push () a method or an array?

Callback Function

A function can be passed as a parameter to another function

```
1  let arr = [1, 2, 3, 4, 5];  
2  
3  arr.forEach(function printVal(val) {  
4    console.log(val);  
5  });  
6
```

```
1  let arr = [1, 2, 3, 4, 5];  
2  
3  arr.forEach((val) => {  
4    console.log(val);  
5  });
```


Callback Function

A function can be passed as a parameter to another function

```
1  let arr = [1, 2, 3, 4, 5];  
2  
3  arr.forEach((val) => {  
4    console.log(val);  
5  });
```

forEach method **executes a callback function** for each array element and passes **three arguments**

<u>Parameter</u>	<u>Meaning</u>
value	item value
index	position
array	whole array

Callback Function

A function can be passed as a parameter to another function

forEach method **executes a callback function** for each array element and passes **three arguments**

```
let nums = [10,20,30];  
  
nums.forEach(function(value, i){  
  console.log(i, value);  
});
```

0 10

1 20

2 30

Callback Function

A function can be passed as a parameter to another function

```
1  let nums = [67, 52, 39];  
2  
3  let calcSquare = (num) => {  
4    console.log(num * num);  
5  };  
6    
7  nums.forEach(calcSquare);  
8
```

A Higher-Order Function (HOF)

A function that either **takes another function as a parameter**
OR **returns a function**.

```
function greet(name){  
    console.log("Hello " + name);  
}  
  
function processUser(callback){  
    let user = "Ali";  
    callback(user);  
}  
  
processUser(greet);
```

- processUser is a Higher-Order
- Function greet is the callback function

Anonymous functions

```
function(parameters) {  
    statements;  
}
```

JS

- ❑ JavaScript allows you to declare anonymous functions
- ❑ quickly creates a function without giving it a name
- ❑ can be stored as a variable, attached as an event handler, etc.

Anonymous function example

```
const greet = function () {  
    console.log("Welcome to GeeksforGeeks!");  
};  
  
greet();
```

```
window.onload = function() {  
    var okButton = document.getElementById("ok");  
    okButton.onclick = okayClick;  
};  
function okayClick() {  
    alert("booyah");  
}
```

JS

Self-Executing Anonymous Functions

Another common use of anonymous functions is to create self-executing functions (also known as IIFE - Immediately Invoked Function Expressions). These functions run immediately after they are defined.

[Example]: Creating a self-executing function.

```
(function () {  
    console.log("Welcome to GeeksforGeeks!");  
})();
```



Recap : Functions

```
// This is a function declaration
function hello() {
  console.log('Hello World');
}

// This is a function expression
var hello = function() {
  console.log('Hello World');
};

// This is also a function expression.
(function() {
  console.log('Hello World');
})();
```

Arrays in JS

Collections of items

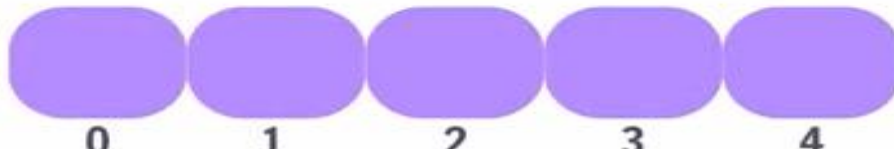
Create Array

```
let heroes = [ "ironman", "hulk", "thor", "batman" ];
```

```
let marks = [ 96, 75, 48, 83, 66 ];
```

Array Indices

arr[0], arr[1], arr[2]



- Example: `color = ["red", "yellow", "green", "blue"];`
- Arrays are zero-based: `color[0]` is "red"

Ways to create an array

- You can use an array literal:
 - **var colors = ["red", "green", "blue"];**
- You can use new Array() to create an empty array:
 - **var colors = new Array();**
 - You can add elements to the array later:
colors[0] = "red"; colors[2] = "blue"; colors[1]="green";
- You can use new Array(n)
 - **var colors = new Array(3);**
 - You can add elements to the array later:
colors[0] = "red"; colors[2] = "blue"; colors[1]="green";
- You can use new Array(...) with two or more arguments to create an array containing those values:
 - **var colors = new Array("red", "green", "blue");**

For ... of loop

Script.js / ...

```
1 let heroes = ["ironman", "thor", "hulk", "shaktiman", "spiderman", "antman"]
2
3 for (let i = 0; i < heroes.length; i++) {
4   console.log(heroes[i]);
5 }
```

for...of is used to:

Directly go through each value (element) of an array or collection.

```
for (variable of array){  
  // code}
```

```
//for of  
for (let hero of heroes) {  
  console.log(hero);  
}
```

let arr = [1,2,3,4];

Method	Syntax Example	What it Does	Output
push()	arr.push(5)	Add element at end	5 (returns new length)
pop()	arr.pop()	Remove last element	4 (returns removed element)
unshift()	arr.unshift(0)	Add at beginning	5 (returns new length)
shift()	arr.shift()	Remove first element	1 (returns value)
forEach()	arr.forEach(n=>console.log(n))	Runs function on each element	prints 1 2 3 4
map()	arr.map(n=>n*2)	Creates new array	[2,4,6,8]
filter()	arr.filter(n=>n%2==0)	Select elements	[2,4] (creates new array)
find()	arr.find(n=>n>2)	First matching element	3 (Stops searching after first match)
sort()	arr.sort()	Sort elements	[1,2,3,4]

let arr = [1,2,3,4];

Method	Syntax Example	What it Does	Output
reduce()	arr.reduce((prev,cur)=>prev = cur)	Combine to one value	10
includes()	arr.includes(3)	Check existence	true
indexOf()	arr.indexOf(3)	Position of element	2
slice()	arr.slice(1,3)	Copy part of array	[2,3] (Start at index 1 Stop BEFORE index 3)
splice()	arr.splice(1,2)	Remove elements	[2,3] (start at index 1 remove 2 elements)
join()	arr.join("-")	Convert to string	"1-2-3-4"custom separator
toString()	arr.toString()	Array → string	"1,2,3,4" (always comma)
reverse()	arr.reverse()	Reverse order	[4,3,2,1]

Reduce

Performs some operations & reduces the array to a single value. It returns that single value.

Example: return largest element

```
let arr = [5, 6, 2, 1, 3];  
  
const output = arr.reduce((prev, curr) => {  
  | return prev > curr ? prev : curr;  
});  
  
console.log(output); ,
```


Practice Questions

Qs. We are given array of marks of students. Filter out of the marks of students that scored 90+.

Qs. Take a number n as input from user. Create an array of numbers from 1 to n.
Use the reduce method to calculate sum of all numbers in the array.
Use the reduce method to calculate product of all numbers in the array.

Practice Questions

Qs. We are given array of marks of students. Filter out of the marks of students that scored 90+.

```
1  let marks = [97, 64, 32, 49, 99, 96, 86];
2
3  let toppers = marks.filter((val) => {
4    |   return val > 90;
5  });
6
7  console.log(toppers);
```

JAVASCRIPT INTERACTION WITH USER

purpose of **alert, confirm, prompt**

JavaScript Popup Boxes

- Alert box
 - User will have to click "OK" to proceed
 - `alert("sometext")`
- Confirm box
 - User will have to click either "OK" or "Cancel" to proceed
 - `confirm("sometext")`
- Prompt box
 - User will have to click either "OK" or "Cancel" to proceed after entering an input value
 - `prompt("sometext","defaultvalue")`

alert(), confirm(), and prompt()

```
let name = prompt("Enter your name:");
```

```
if (confirm("Is your name " + name + "?")) {  
  alert("Welcome " + name);  
} else {  
  alert("Please refresh and enter again");  
}
```

Enter your name:

Ahmed|

OK

Cancel

Is your name Ahmed?

OK

Cancel

Welcome Ahmed

OK

Please refresh and enter again

OK

alert() and confirm()

```
alert("Text to be displayed");
```

- Display a message in a dialog box.
- The script pauses until the user presses OK.

```
let answer = confirm("Are you sure?");
```

- Display a message in a dialog box with two buttons: "OK" or "Cancel".
- confirm() returns **true** when the user clicks OK and **false** when the user clicks Cancel.

prompt ()

```
let id = prompt("What is your student id number?");  
let name = prompt("What is your name?", "No name");
```

- Display a message and allow the user to enter a value
- The second argument is the "default value" to be displayed in the input textfield.
- If no default value is provided, the input field is empty
- If the user click the "OK" button, **prompt ()** returns the value in the input textfield as a string.
- If the user click the "Cancel" button, **prompt ()** returns null.

JavaScript Language

- Conditional statement
 - > if, if.. else, switch
- Loop
 - > for loop, while loop
- try...catch
- throw

All have the same syntax as those found in C and Java.

JavaScript Functions

- A function is a reusable block of code that runs when it is called (invoked).
- Functions may be triggered by
 - direct calls
 - user events (click, submit, etc.)
- Code inside a function does not execute automatically when the page loads
- Functions can be called from anywhere in the script, and functions in external .js files can be reused across multiple pages.
- Best practice: place JavaScript in an external file and link it using: `<script src="script.js" defer></script>`

Browser Object Model

JavaScript is not only a language
it is a language that controls the browser

- When a webpage opens, the browser does not just show HTML.
- The browser creates objects in memory representing the page.
- JavaScript talks to those objects.
- JavaScript does NOT directly talk to HTML. It talks to browser objects.

Why Do We Need DOM?

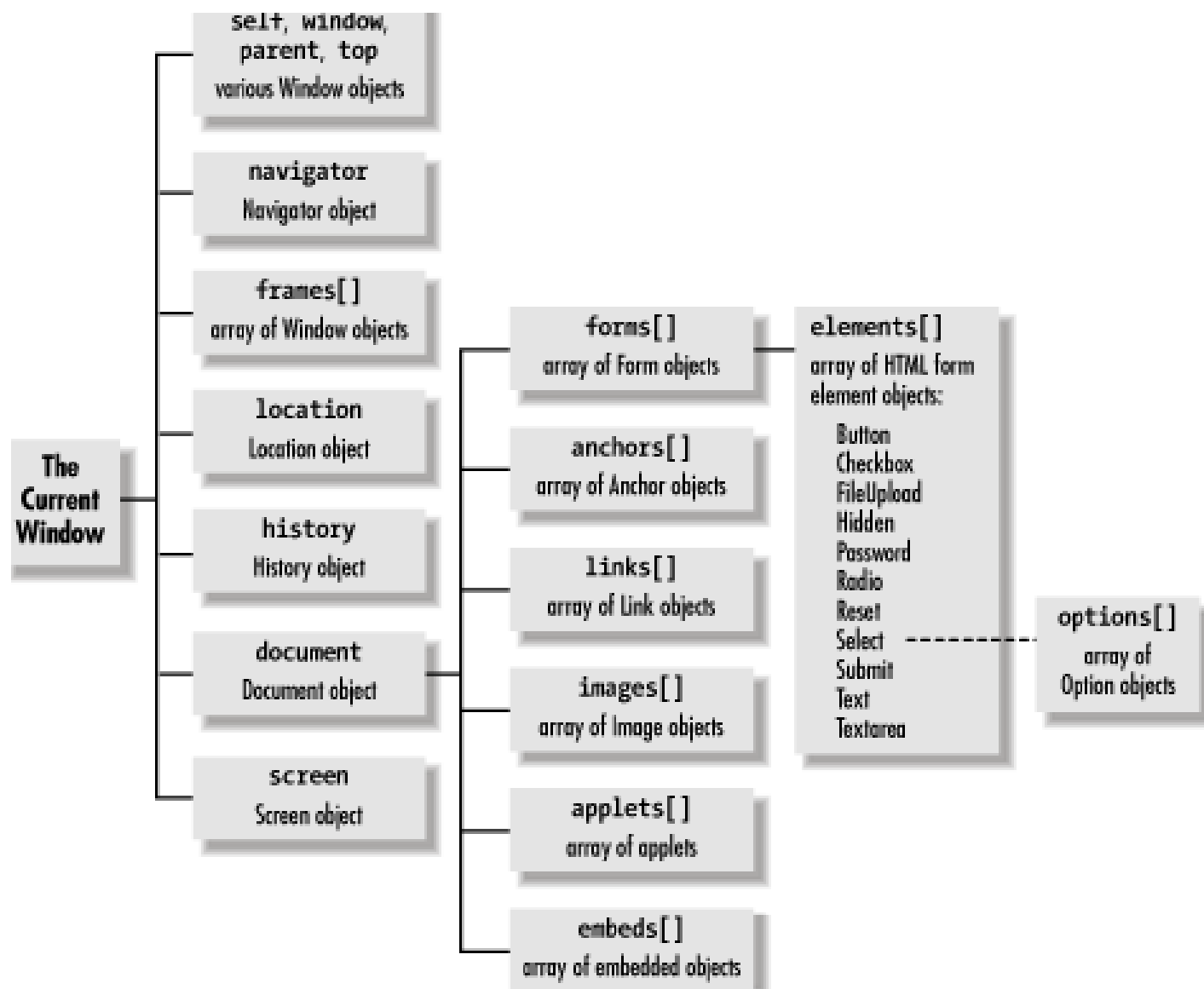
Without DOM:

- ✗ JavaScript cannot access HTML
- ✗ Cannot change text
- ✗ Cannot change styles
- ✗ Cannot respond to events

With DOM:

- ✓ Change content
- ✓ Change styles
- ✓ Add/remove elements
- ✓ Handle clicks

The Browser Object Hierarchy



The Browser Objects (Global DOM)

name	description
document	current HTML page and its content
history	list of pages the user has visited
location	URL of the current HTML page
navigator	info about the web browser you are using
screen	info about the screen area occupied by the browser
window	the browser window

The window object

The window object represents an open window in a browser. It is browser's object (not JavaScript's) & is automatically created by browser.

It is a **global** object with lots of properties & methods.

- *the entire browser window (DOM top-level object)*
- technically, all global code and variables become part of the window object properties:
 - document, history, location, name
- methods:
 - alert, confirm, prompt (**popup boxes**)
 - setInterval, setTimeout, clearInterval, clearTimeout (**timers**)
 - open, close (**popping up new browser windows**)
 - blur, focus, moveBy, moveTo, print, resizeBy, resizeTo, scrollBy, scrollTo

Window object

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<button onclick="goBack()">Go  
  Back</button>
```

```
</body>
```

```
</html>
```

```
<script>
```

```
function goBack() {  
  window.history.back();  
}
```

```
</script>
```



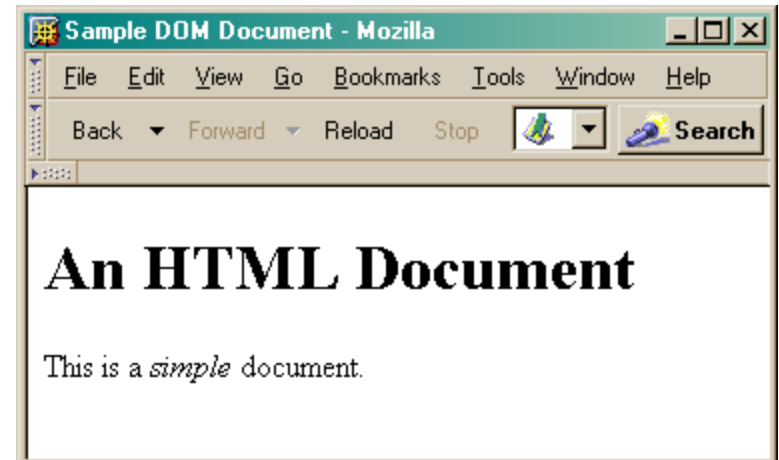
Document Object Model

- Document Object Model
 - Your web browser builds a *model* of the web page (the *document*) that includes all the *objects* in the page (tags, text, etc)
 - All of the properties, methods, and events available to the web developer for manipulating and creating web pages are organized into objects
 - Those objects are accessible via scripting languages in modern web browsers

This is what the browser reads

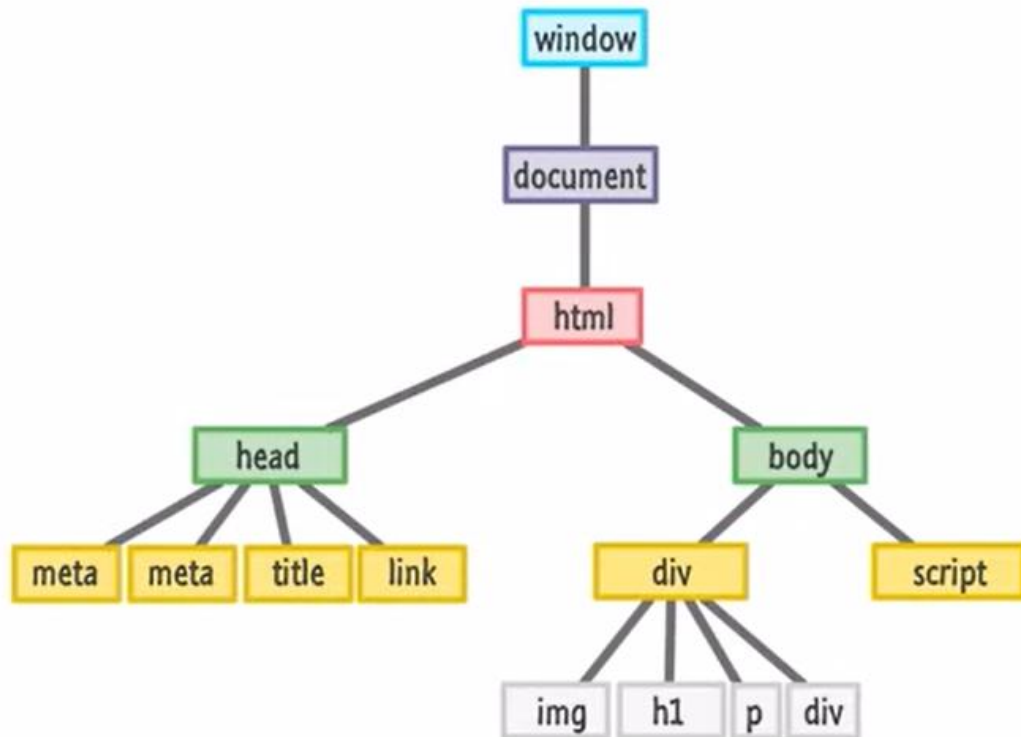
```
<html>
  <head>
    <title>Sample DOM Document</title>
  </head>
  <body>
    <h1>An HTML Document</h1>
    <p>This is a <i>simple</i> document.
  </body>
</html>
```

This is what the browser displays on screen.



Document Object Model (DOM)

When a web page is loaded, the browser creates a **Document Object Model (DOM)** of the page

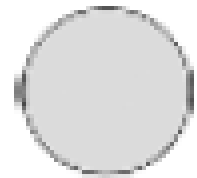
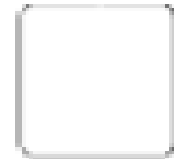
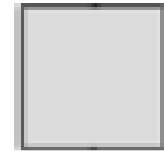


Types of DOM nodes

```
<p>  
This is a paragraph of text with a  
<a href="/path/page.html">link in it</a>.  
</p>
```

HTML

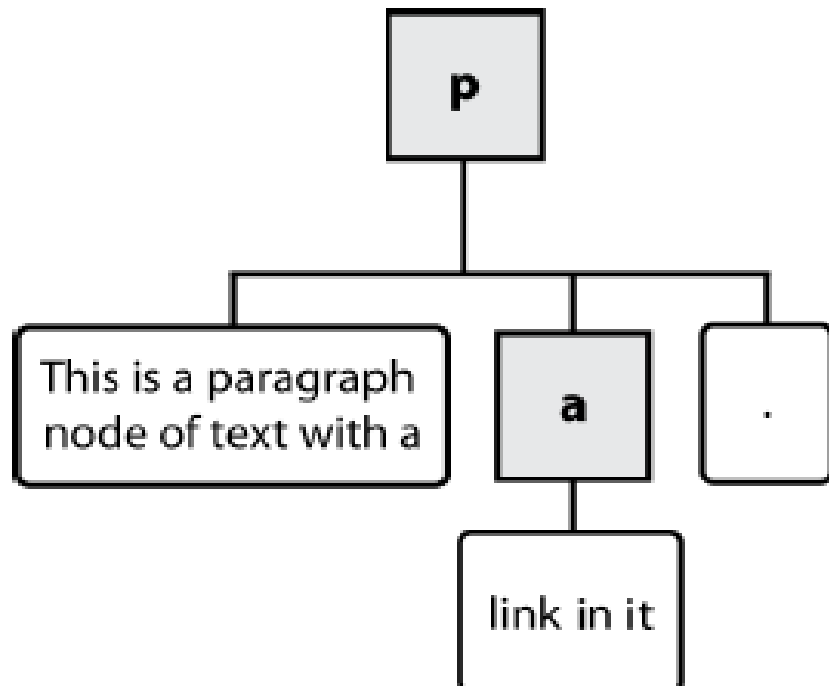
- element nodes (HTML tag)
 - can have children and/or attributes
- text nodes (text in a block element)
- attribute nodes (attribute/value pair)
 - text/attributes are children in an element
 - cannot have children or attributes
 - not usually shown when drawing the DOM tree



Types of DOM nodes

```
<p>  
This is a paragraph of text with a  
<a href="/path/page.html">link in it</a>.  
</p>
```

HTML



HTML DOM

- HTML DOM methods are **actions** you can perform (on HTML Elements)
 - add or deleting an HTML element
- HTML DOM properties are **values** (of HTML Elements) that you can set or change
 - changing the content of an HTML element

- `html>`
`<body>`

`<p id="demo"></p>`

`<script>`
`document.getElementById("demo").innerHTML = "Hello World!";`
`</script>`

`</body>`
`</html>`

Document Object: Write text to the output

```
<html>  
  <body>  
    <script type="text/javascript">  
      document.write("Hello World!")  
    </script>  
  </body>  
</html>
```

- **Document Object: Write text with Formatting to the output**

```
<html>
```

```
  <body>
```

```
    <script type="text/javascript">
```

```
      document.write("<h1>Hello World!</h1>")
```

```
    </script>
```

```
  </body>
```

```
</html>
```

Some information from a document

```
<html>
  <head>
    <title>DOM Sample 1</title>
  </head>
  <body>
    Information about this document.<br>
    <script type="text/javascript">
      document.write("<br>Title: ",document.title);
      document.write("<br>Referrer: ",document.referrer);
      document.write("<br>Domain: ",document.domain);
      document.write("<br>URL: ",document.URL);
    </script>
  </body>
</html>
```


Finding HTML Elements

Selecting with id

```
document.getElementById("myId")
```

Selecting with class

```
document.getElementsByClassName("myClass")
```

Selecting with tag

```
document.getElementsByTagName("p")
```

Finding HTML Elements

Query Selector

```
document.querySelector("myId / myClass / tag")
```

//returns first element

```
document.querySelectorAll("myId / myClass / tag")
```

//returns a NodeList

JavaScript function to get the values of First and Last name of form.

```
<body>
<form id="form1" onsubmit= "return getFormvalue()">
First name: <input type="text" name="fname" value="Ahmad"><br>
Last name: <input type="text" name="lname" value="Ali"><br>
<input type="submit" value="Submit">
</form> </body> </html>
```

```
function getFormvalue() {
var firstname = (document.getElementsByName("fname"))[0].value;
var secondname = (document.getElementsByName("lname"))[0].value;
alert(firstname+ " " + secondname);
}
```

Document Object: Use getElementById()

```
<html>
  <head>
    <script type="text/javascript">
      function getElement() {
        var x=document.getElementById("myHeader")
        alert("I am a " + x.tagName + " element")    }
    </script> </head>
  <body>
    <h1 id="myHeader" onclick="getElement()">Click to see
    what element I am!</h1>
  </body>
</html>
```

Modifying DOM

Properties

- **tagName** : returns tag for element nodes
- **innerText** : returns the text content of the element and all its children
- **innerHTML** : returns the plain text or HTML contents in the element
- **textContent** : returns textual content even for hidden elements



```

<body>
  <div>
    <h3>Fruits</h3>
    <ul>
      <li>mango</li>
      <li>orange</li>
      <li>litchi</li>
    </ul>
  </div>
  <script src="script.js"></script>

```

Fruits

- mango
- orange
- litchi

```

> div script.js:2
> div.innerText
'Fruits\nmango\norange\nlitchi'
> div.innerHTML
'\n    <h3>Fruits</h3>\n    <ul>\n      <li>mango</li>\n      <li>orange</li>\n      <li>litchi</li>\n    </ul>\n'

```

Fruits

- mango
- orange
- litchi

```
8 </head>
9 <body>
10   <div>
11     <h3>Fruits</h3>
12     <ul>
13       <li>mango</li>
14       <li>orange</li>
15       <li>litchi</li>
16     </ul>
17   </div>
18   <script src="script.js"></script>
19 </body>
```

► div

script.js:2

> div.innerText

< 'Fruits\nmango\norange\nlitchi'

> div.innerHTML

< '\n <h3>Fruits</h3>\n \n mango\n orange\n litchi\n \n '

heading

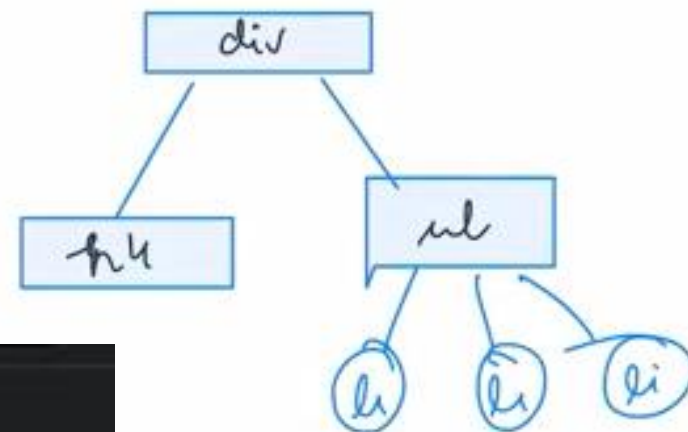
<h1 style="visibility: hidden">old Heading</h1>

heading.innerText

''

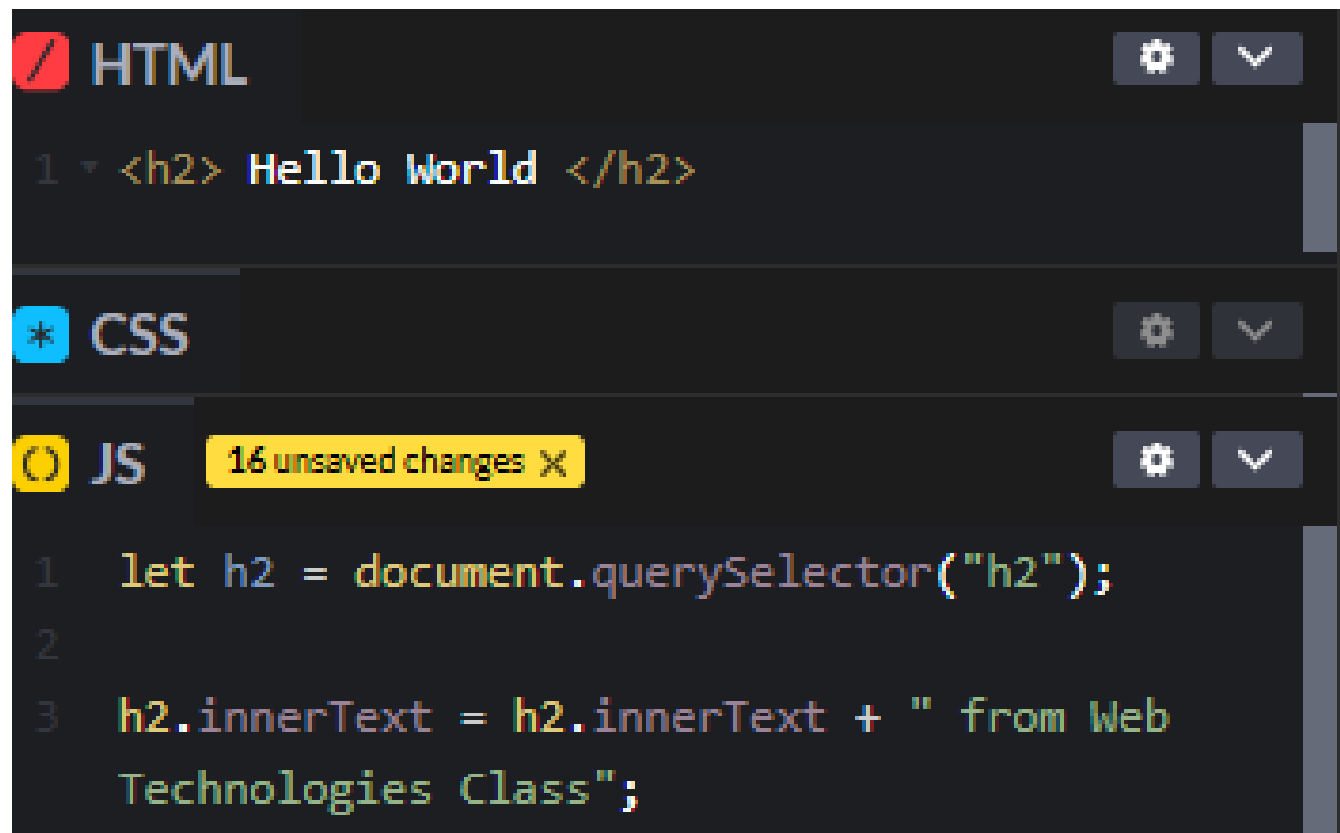
heading.textContent

'old Heading'



Practice Question

- Create a H2 heading element with text – “Hello JavaScript”. Append “from Web Technologies Class” to this text using JS.



The screenshot shows a web development IDE with three panels: HTML, CSS, and JS. The HTML panel contains a single line of code: `<h2> Hello World </h2>`. The CSS panel is empty. The JS panel contains two lines of code: `let h2 = document.querySelector("h2");` and `h2.innerText = h2.innerText + " from Web Technologies Class";`. The JS panel also shows a notification for "16 unsaved changes".

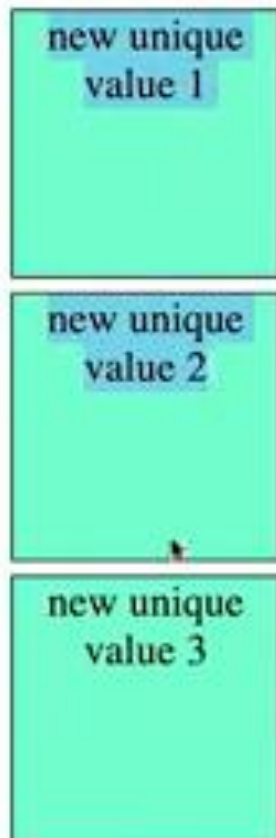
```
HTML
1 <h2> Hello World </h2>

CSS

JS 16 unsaved changes X
1 let h2 = document.querySelector("h2");
2
3 h2.innerText = h2.innerText + " from Web
  Technologies Class";
```


Practice Question

- Create 3 divs with common class name – “box”. Access them & add some unique text to each of them.



```
let divs = document.querySelectorAll(".box");  
divs[0].innerText = "new unique value 1";  
divs[1].innerText = "new unique value 2";  
divs[2].innerText = "new unique value 3";
```

```
let idx = 1;  
for (div of divs) {  
  div.innerText = `new unique value ${idx}`;  
  idx++;  
}
```

Example

```
<html>
```

```
<body>
```

```
<p id="intro">Hello World!</p>
```

```
<p>This example demonstrates the <b>getElementById</b> method!</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
var myElement = document.getElementById("intro");
```

```
document.getElementById("demo").innerHTML =
```

```
"The text from the intro paragraph is " + myElement.innerHTML;
```

```
</script>
```

```
</body>
```

```
</html>
```

Hello World!

This example demonstrates the

getElementById method!

The text from the intro paragraph is

Hello World!